



My Git Cheat Sheet

A complete, no-nonsense reference for Git — from your very first commit to advanced workflows. Bookmark it. Use it daily.



Table of Contents

- [Setup & Config](#)
 - [Starting a Repo](#)
 - [Staging & Committing](#)
 - [Branching](#)
 - [Merging & Rebasing](#)
 - [Remote Repositories](#)
 - [Undoing Changes](#)
 - [Stashing](#)
 - [Viewing History & Diffs](#)
 - [Tagging](#)
 - [Advanced Commands](#)
 - [Git Aliases \(Shortcuts\)](#)
 - [Day-to-Day Workflow Examples](#)
-



Setup & Config

First things first — tell Git who you are.

```
# Set your name and email (stored globally)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Set VS Code as default editor
git config --global core.editor "code --wait"

# Set main as default branch name
git config --global init.defaultBranch main

# View all your config settings
```

```
git config --list
```

```
# View a specific setting
```

```
git config user.name
```



Starting a Repo

```
# Initialize a new Git repo in the current folder
```

```
git init
```

```
# Clone an existing repo from GitHub/GitLab/etc.
```

```
git clone https://github.com/user/repo.git
```

```
# Clone into a specific folder name
```

```
git clone https://github.com/user/repo.git my-folder
```

```
# Clone only the latest snapshot (faster for large repos)
```

```
git clone --depth 1 https://github.com/user/repo.git
```



Staging & Committing

The **staging area** is your prep zone before committing.

```
# Check what's changed / what's staged
```

```
git status
```

```
# Stage a specific file
```

```
git add filename.txt
```

```
# Stage everything in current directory
```

```
git add .
```

```
# Stage parts of a file interactively (pick chunks)
```

```
git add -p filename.txt
```

```
# Commit staged changes with a message
```

```
git commit -m "Add login feature"
```

```
# Stage all tracked files AND commit in one shot
```

```
git commit -am "Fix typo in navbar"
```

```
# Amend the last commit (message or files)
```

```
git commit --amend -m "Better commit message"
```

```
# Commit with a detailed multi-line message
```

```
git commit
```

```
# (Opens your editor – first line = title, blank line, then details)
```

💡 **Tip:** Write commit messages in imperative mood – *"Fix bug"* not *"Fixed bug"*.

Branching

Branches let you work on features without touching the main code.

```
# List all local branches
```

```
git branch
```

```
# List all branches (local + remote)
```

```
git branch -a
```

```
# Create a new branch
```

```
git branch feature/login
```

```
# Switch to a branch
```

```
git checkout feature/login
```

```
# Create AND switch in one command (modern way)
```

```
git switch -c feature/login
```

```
# Rename current branch
```

```
git branch -m new-name
```

```
# Delete a branch (safe – only if merged)
```

```
git branch -d feature/login
```

```
# Force delete a branch
```

```
git branch -D feature/login
```

```
# See which branch each commit is on
```

```
git log --oneline --graph --all
```

Merging & Rebasing

```
# Merge a branch into your current branch
git merge feature/login

# Merge without fast-forward (keeps merge commit)
git merge --no-ff feature/login

# Abort a merge that has conflicts
git merge --abort

# Rebase current branch onto main
git rebase main

# Interactive rebase - edit last 3 commits
git rebase -i HEAD~3
# Options: pick, squash, reword, drop, edit

# Abort a rebase
git rebase --abort

# Continue after resolving conflicts
git rebase --continue
```

Merge vs Rebase:

- **merge** → preserves history, adds a merge commit
- **rebase** → rewrites history, cleaner linear log (don't use on shared branches)

Remote Repositories

```
# See your remotes
git remote -v

# Add a remote
git remote add origin https://github.com/user/repo.git

# Rename a remote
git remote rename origin upstream
```

```
# Remove a remote
git remote remove origin

# Fetch changes (download but don't merge)
git fetch origin

# Pull = fetch + merge
git pull origin main

# Pull with rebase instead of merge
git pull --rebase origin main

# Push your branch to remote
git push origin feature/login

# Push and set upstream (first time)
git push -u origin feature/login

# Force push (⚠ use carefully — rewrites remote history)
git push --force-with-lease origin feature/login

# Delete a remote branch
git push origin --delete feature/login
```

Undoing Changes

```
# Discard changes in working directory (unstaged)
git restore filename.txt

# Discard ALL unstaged changes
git restore .

# Unstage a file (keep changes in working dir)
git restore --staged filename.txt

# Undo last commit but keep changes staged
git reset --soft HEAD~1

# Undo last commit and unstage changes
git reset --mixed HEAD~1
```

```
# Undo last commit and DISCARD all changes (⚠ destructive)
git reset --hard HEAD~1

# Safely undo a commit by creating a new "undo" commit
git revert HEAD

# Revert a specific commit
git revert abc1234

# Remove untracked files (preview first)
git clean -n

# Remove untracked files (actually delete)
git clean -f

# Remove untracked files AND directories
git clean -fd
```

Stashing

Stash saves your work-in-progress without committing.

```
# Stash current changes
git stash

# Stash with a description
git stash push -m "WIP: half-done login form"

# List all stashes
git stash list

# Apply most recent stash (keeps it in stash list)
git stash apply

# Apply a specific stash
git stash apply stash@{2}

# Apply AND remove from stash list
git stash pop

# Delete a specific stash
git stash drop stash@{1}
```

```
# Clear all stashes
git stash clear

# Stash including untracked files
git stash -u
```

Viewing History & Diffs

```
# Full commit log
git log

# Compact one-line log
git log --oneline

# Visual branch graph
git log --oneline --graph --all

# Log for a specific file
git log --oneline filename.txt

# Show changes in last commit
git show

# Show a specific commit
git show abc1234

# Diff: unstaged changes
git diff

# Diff: staged changes (about to be committed)
git diff --staged

# Diff between two branches
git diff main feature/login

# Diff between two commits
git diff abc1234 def5678

# Who changed what line - blame
git blame filename.txt
```

```
# Search all commits for a keyword
git log --all --oneline -S "search term"
```

Tagging

Tags mark specific points in history (e.g., releases).

```
# Create a lightweight tag
git tag v1.0.0

# Create an annotated tag (recommended)
git tag -a v1.0.0 -m "First stable release"

# List all tags
git tag

# Tag a specific past commit
git tag -a v0.9.0 abc1234 -m "Beta release"

# Push a tag to remote
git push origin v1.0.0

# Push all tags
git push origin --tags

# Delete a local tag
git tag -d v1.0.0

# Delete a remote tag
git push origin --delete v1.0.0
```

Advanced Commands

Cherry-pick – apply a single commit from another branch

```
git cherry-pick abc1234
```



```
# Cherry-pick a range of commits
git cherry-pick abc1234^..def5678
```

Bisect – find which commit introduced a bug

```
git bisect start
git bisect bad                # current commit is broken
git bisect good v1.0.0        # this version worked
# Git checks out middle commit – test it, then:
git bisect good                # or: git bisect bad
# Repeat until Git finds the culprit
git bisect reset              # done, go back to HEAD
```

Worktree – check out multiple branches at once

```
git worktree add ../hotfix hotfix/urgent-bug
# Now you have two working directories simultaneously
git worktree list
git worktree remove ../hotfix
```

Reflog – your safety net (tracks every HEAD movement)

```
git reflog
# Find a lost commit and restore it
git checkout abc1234
```

Git Aliases (Shortcuts)

Add these to your `~/.gitconfig` under `[alias]` :

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg "log --oneline --graph --all --decorate"
git config --global alias.undo "reset --soft HEAD~1"
git config --global alias.unstage "restore --staged"
```

```
# Usage
git st          # = git status
git lg          # = beautiful branch graph
git undo        # = undo last commit, keep changes
```



Day-to-Day Workflow Examples



Start a new feature

```
git switch main
git pull origin main
git switch -c feature/user-profile
# ... do your work ...
git add .
git commit -m "Add user profile page"
git push -u origin feature/user-profile
# Open a Pull Request on GitHub
```



Quick hotfix on production

```
git switch main
git pull origin main
git switch -c hotfix/fix-crash
# ... fix the bug ...
git commit -am "Fix null pointer crash on login"
git push -u origin hotfix/fix-crash
# Merge to main, then:
git tag -a v1.2.1 -m "Patch: fix login crash"
git push origin --tags
```



Keep your branch up to date with main

```
git switch feature/my-feature
git fetch origin
git rebase origin/main
# Resolve any conflicts, then:
git rebase --continue
git push --force-with-lease origin feature/my-feature
```

💣 Oops – committed to the wrong branch

```
git log --oneline -3          # note the commit hash
git reset --soft HEAD~1      # undo commit, keep changes staged
git switch correct-branch
git commit -m "Your original message"
```

🧹 Clean up before a PR

```
# Squash last 3 commits into one
git rebase -i HEAD~3
# Change 'pick' to 'squash' on 2nd and 3rd lines
# Edit the combined commit message
git push --force-with-lease origin feature/my-feature
```

📁 .gitignore Essentials

```
# Dependencies
node_modules/
vendor/

# Build output
dist/
build/
*.o
*.pyc

# Environment & secrets
.env
.env.local
*.pem
*.key

# OS files
.DS_Store
Thumbs.db
```

```
# IDE
.vscode/
.idea/
*.swp
```

💡 Generate `.gitignore` for your stack: gitignore.io

Emergency Reference

Situation	Command
Undo last commit (keep changes)	<code>git reset --soft HEAD~1</code>
Discard all local changes	<code>git restore .</code>
Recover a deleted branch	<code>git reflog</code> → <code>git checkout <hash></code>
Stop tracking a committed file	<code>git rm --cached filename</code>
Fix last commit message	<code>git commit --amend -m "New message"</code>
See what changed in a file	<code>git log -p filename.txt</code>
Find who broke something	<code>git bisect start</code>
Save work without committing	<code>git stash push -m "WIP"</code>

Made with ❤️ for developers who type `git push` and pray 🙏

★ **Star this repo** if it saved your day!